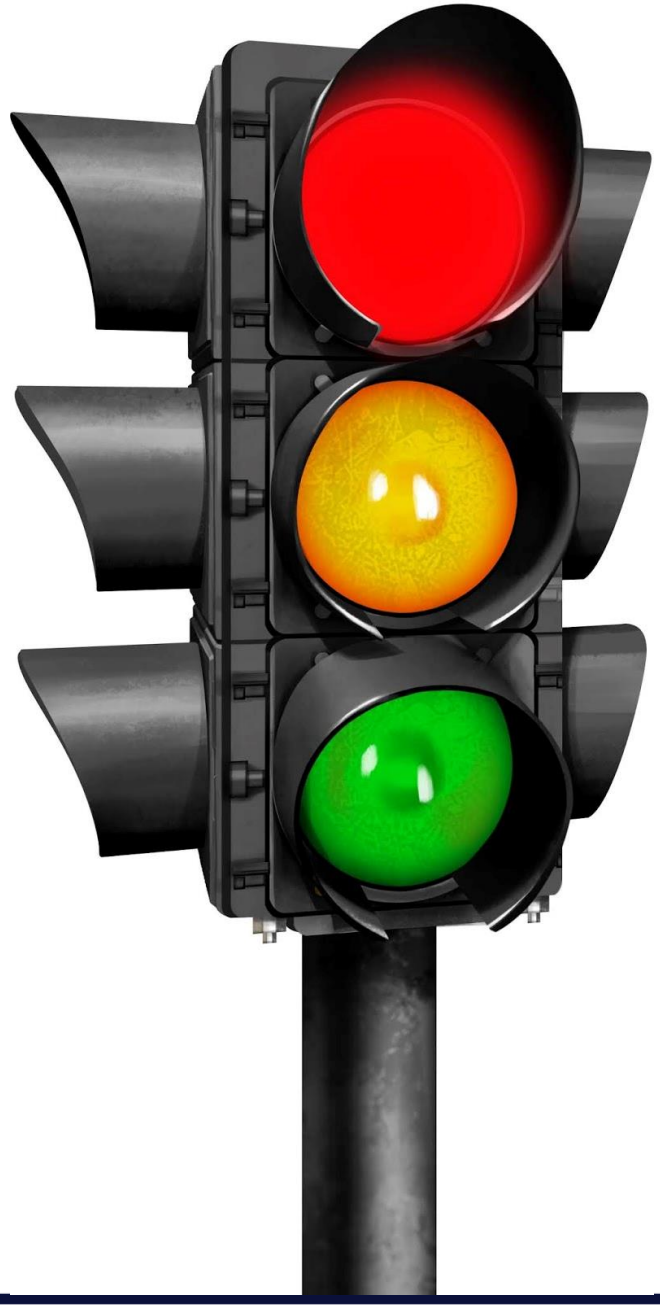
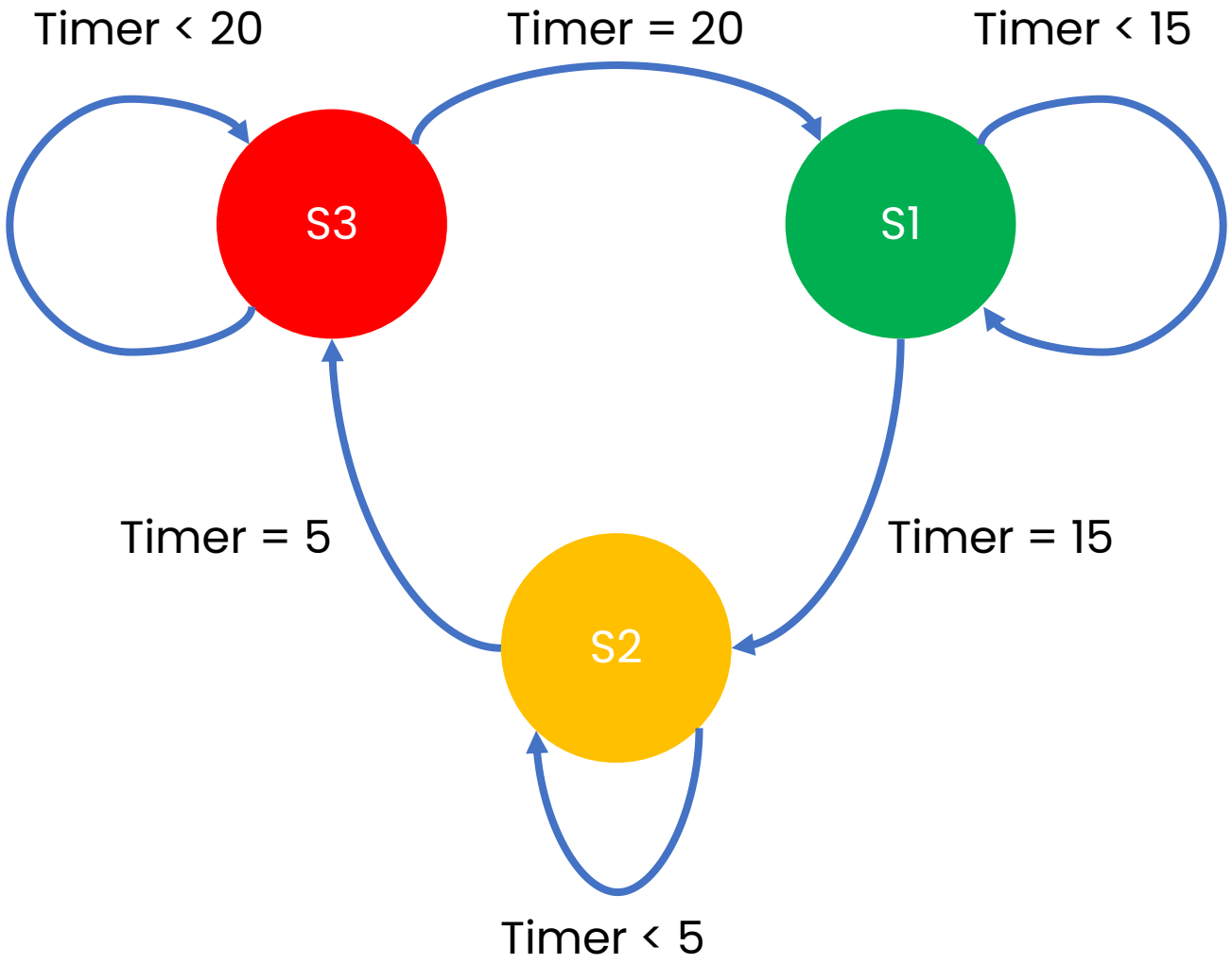
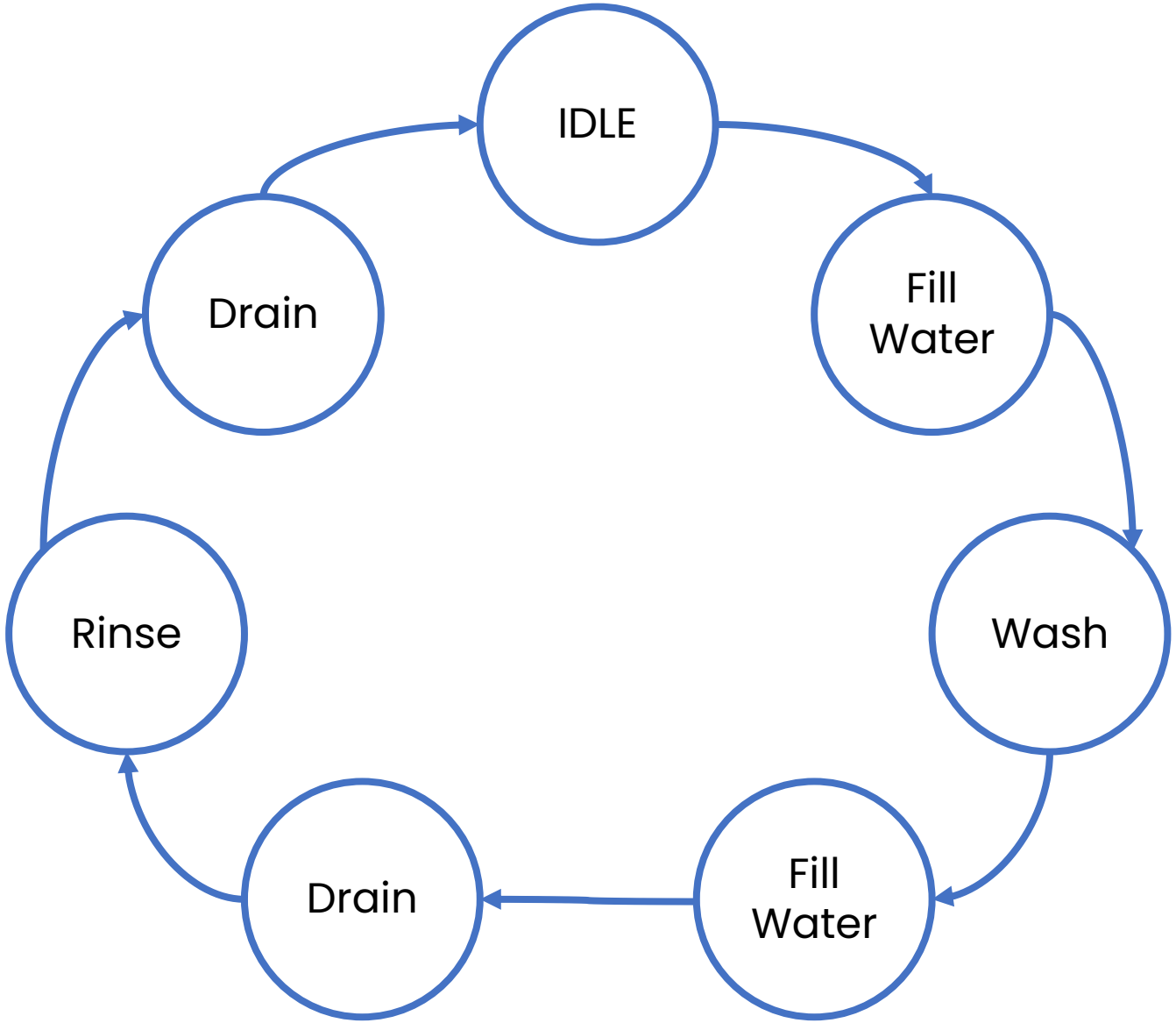
Session 4 – Verilog (FSM)

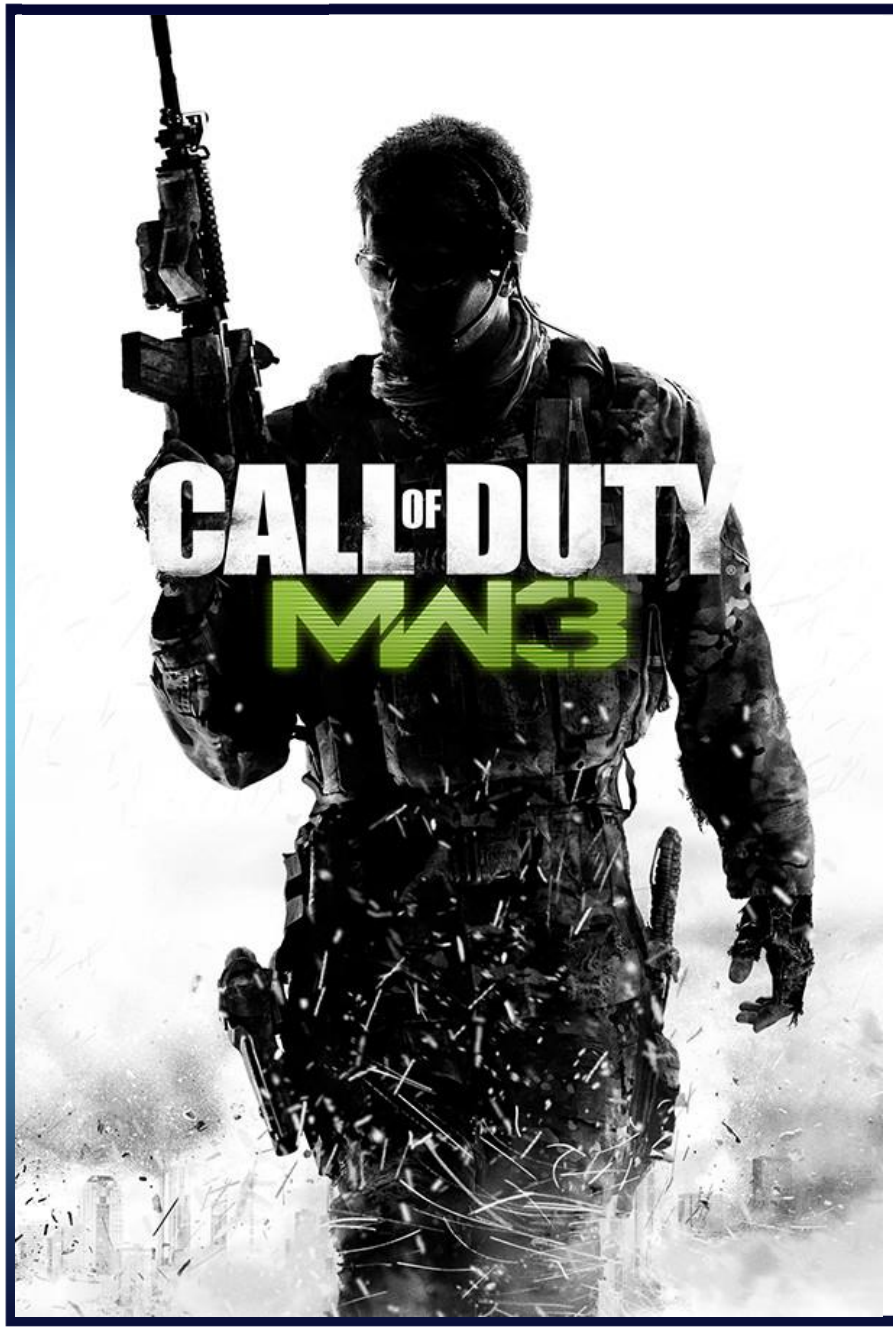
Digital Design:
Verilog and FPGA Course

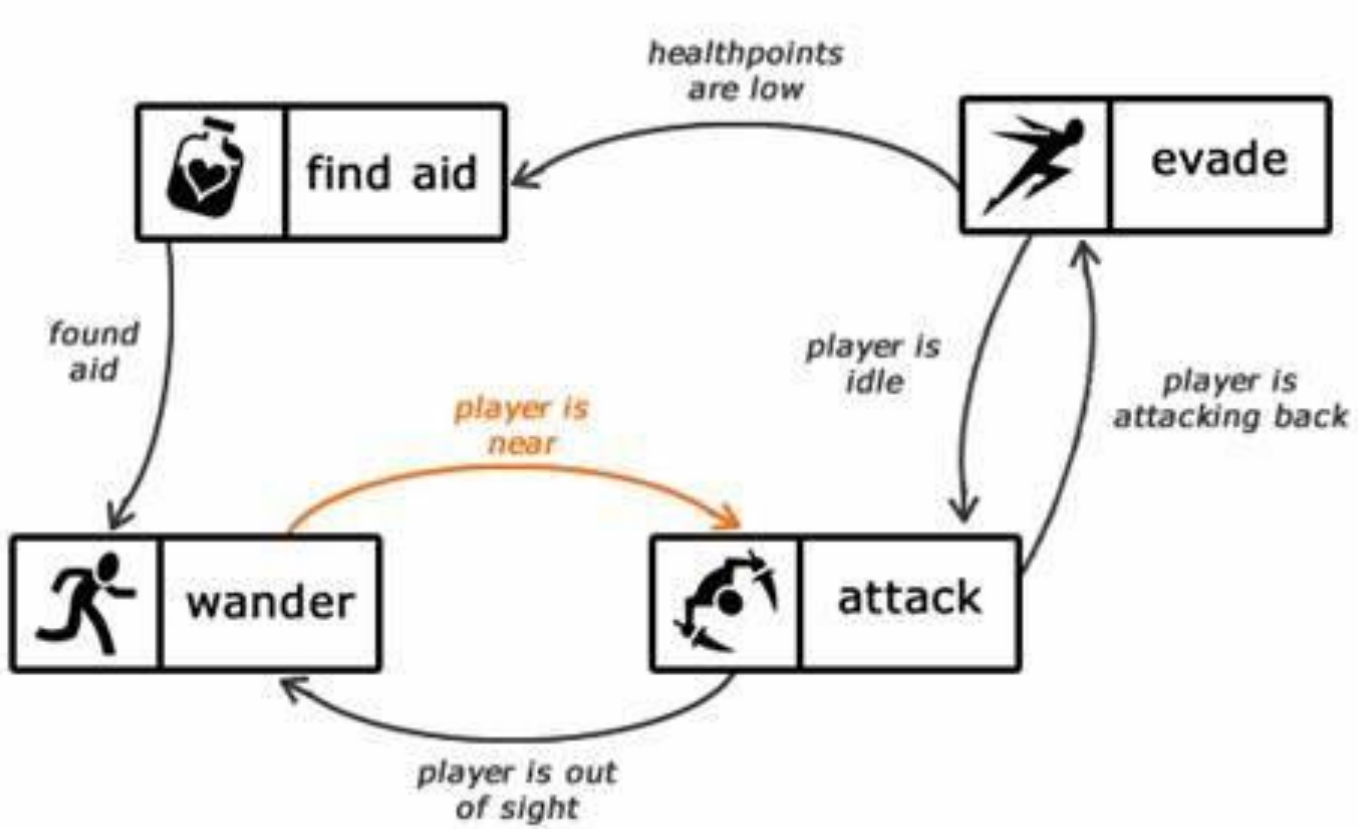






Digital Design:
Verilog and FPGA Course



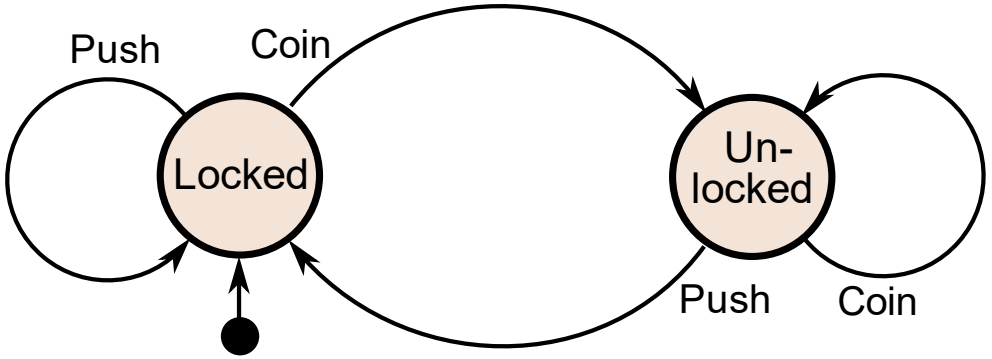


Digital Design:
Verilog and FPGA Course

Finite State Machine (FSM)

A finite state machine (FSM) is a mathematical model used to represent and control the behavior of systems that can exist in a finite number of states at any given time. It is a concept widely used in computer science, engineering, and other fields to analyze, design systems with discrete and sequential behavior.

An FSM consists of a set of states, a set of input events or stimuli, a set of output actions or responses, and a set of transitions between states based on input events. It can be visualized as a directed graph where nodes represent the states and the edges connecting the nodes represent transitions.



Digit
Verilog and FPGA Course

Analysis of FSM

Steps to analyze an FSM

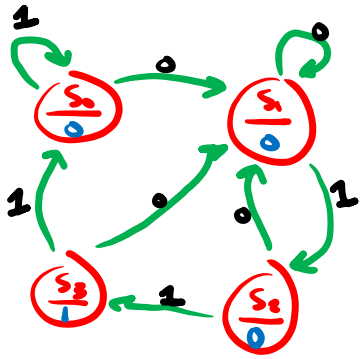
State Equation

$$A(t+1) = (A \oplus B) \cdot X$$

$$B(t+1) = A\bar{B} + \bar{X}$$

$$y = AB$$

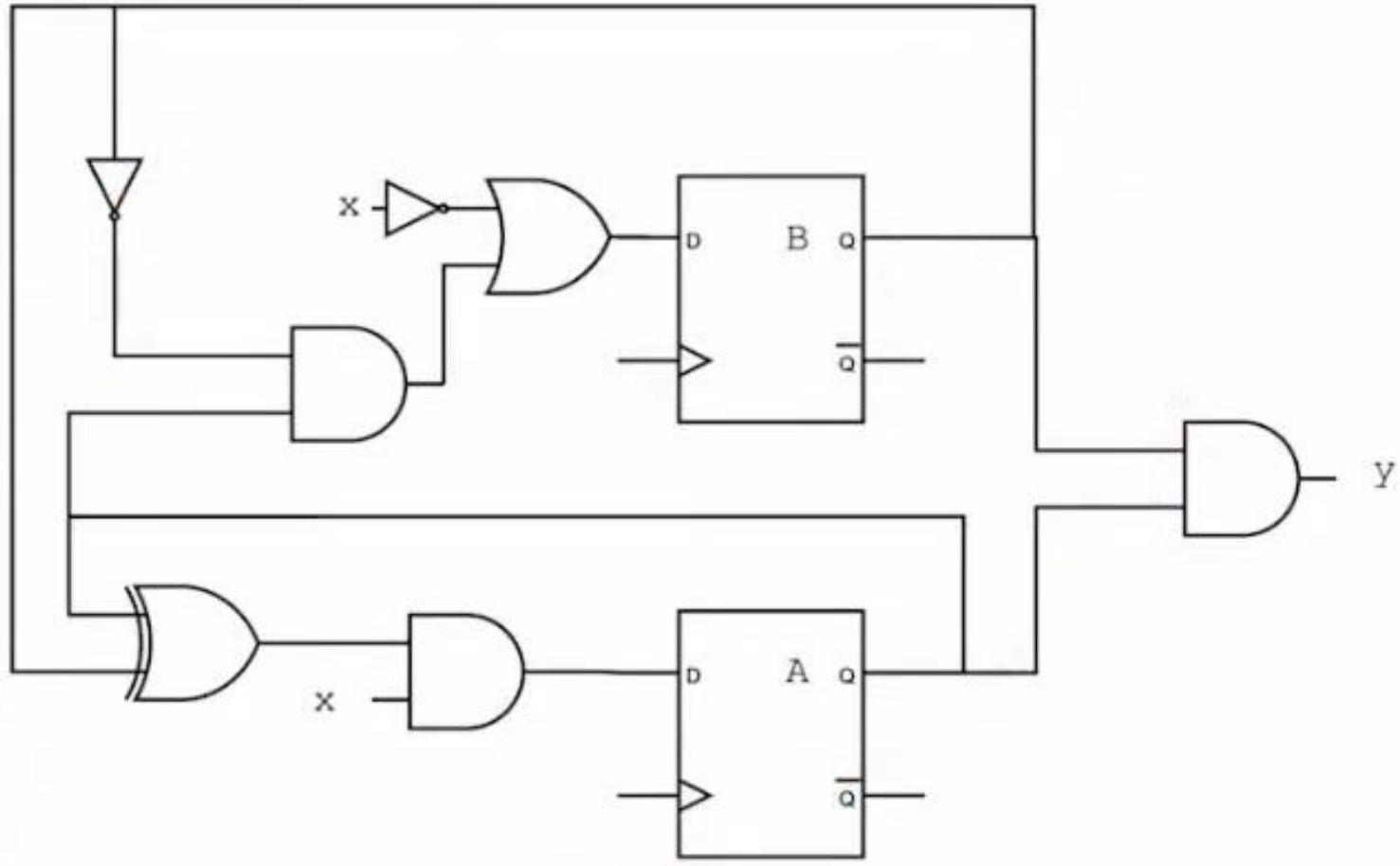
State Diagram



State Table

Present State		Next State X = 0		Next State X = 1		Output
A	B	A	B	A	B	Y
0	0	0	1	0	0	0
0	1	0	1	1	0	0
1	0	0	1	1	1	0
1	1	0	1	0	0	1

Example 1

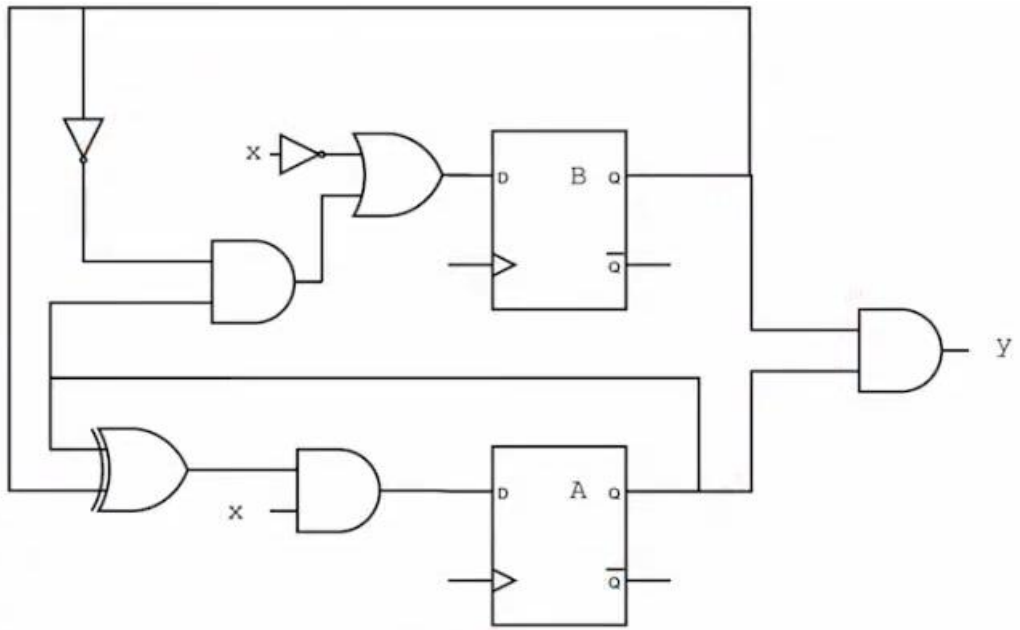


State Equation

$$A(t+1) = (A \oplus B) \cdot X$$

$$B(t+1) = A\bar{B} + \bar{X}$$

$$y = AB$$



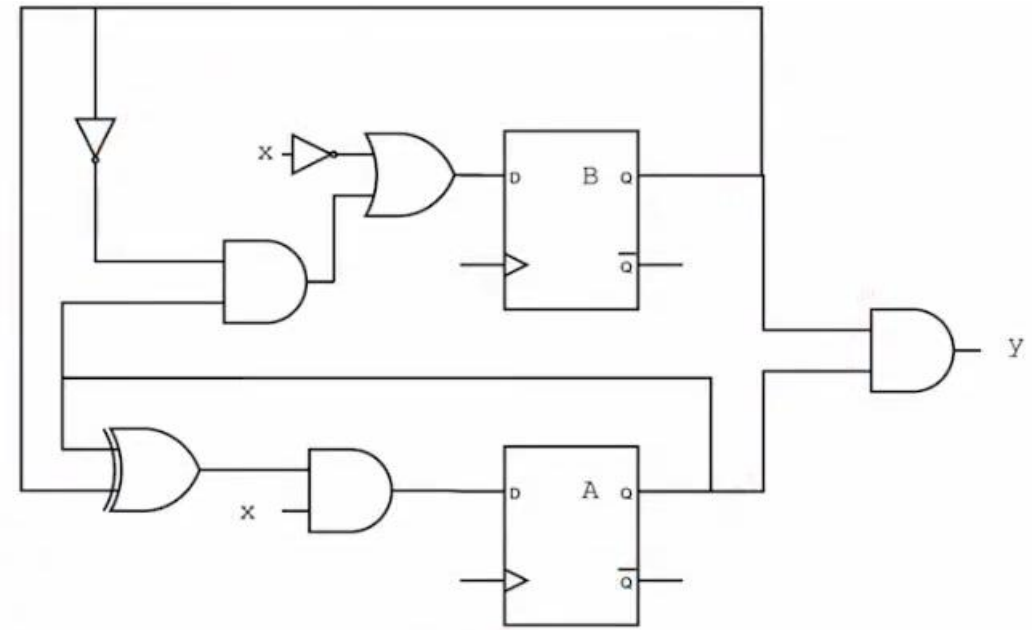
State Table

$$A(t+1) = (A \oplus B) \cdot X$$

$$B(t+1) = A\bar{B} + \bar{X}$$

$$y = AB$$

Present State		Input	Next State		Output
A	B	X	A	B	Y



State Table

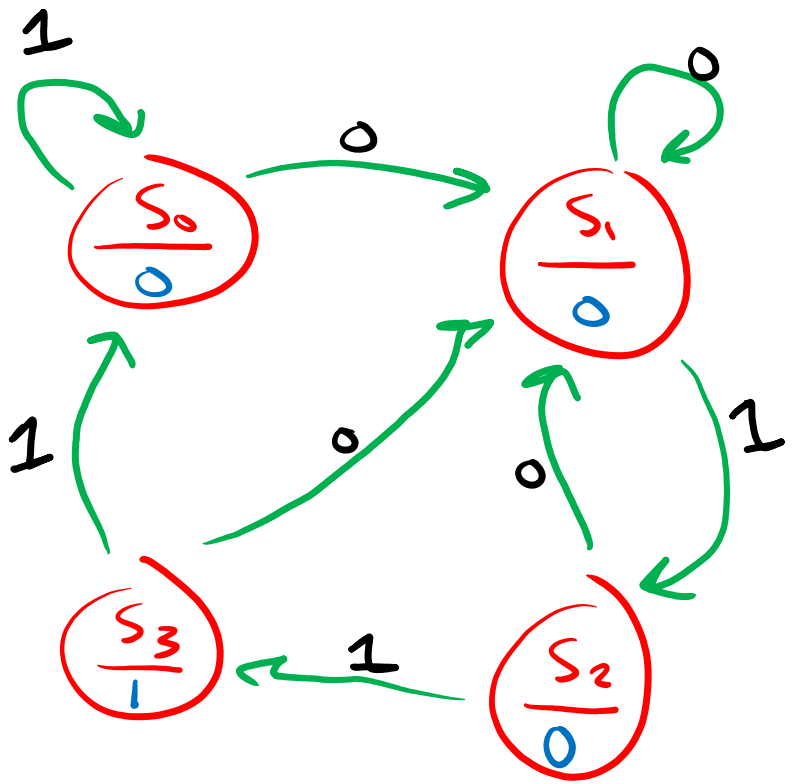
Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	1	1	1	0
1	1	0	0	1	1
1	1	1	0	0	1

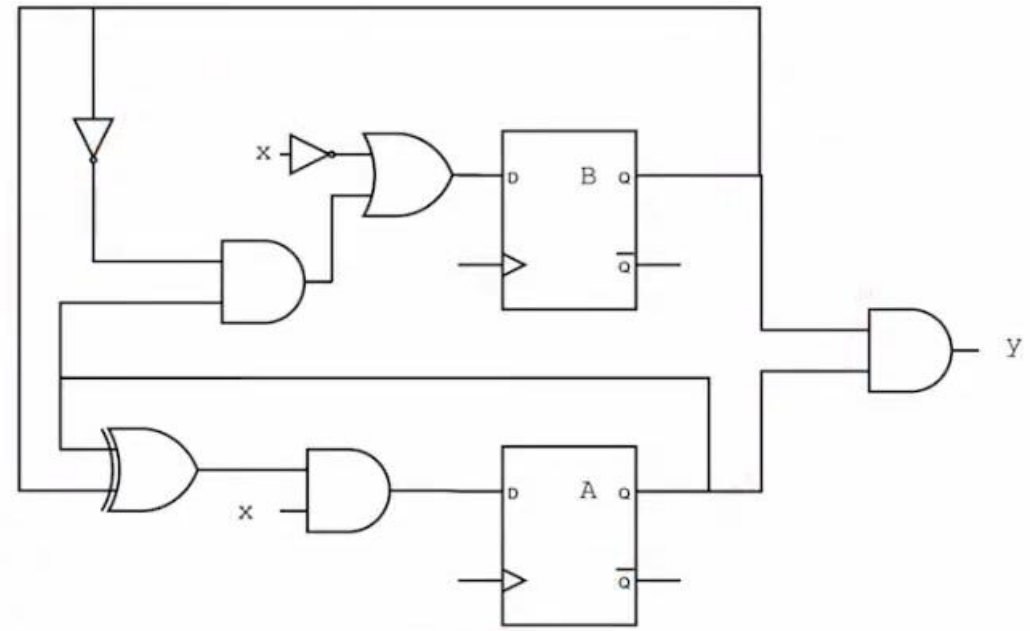
Present State		Next State X = 0		Next State X = 1		Output
A	B	A	B	A	B	Y

State Diagram

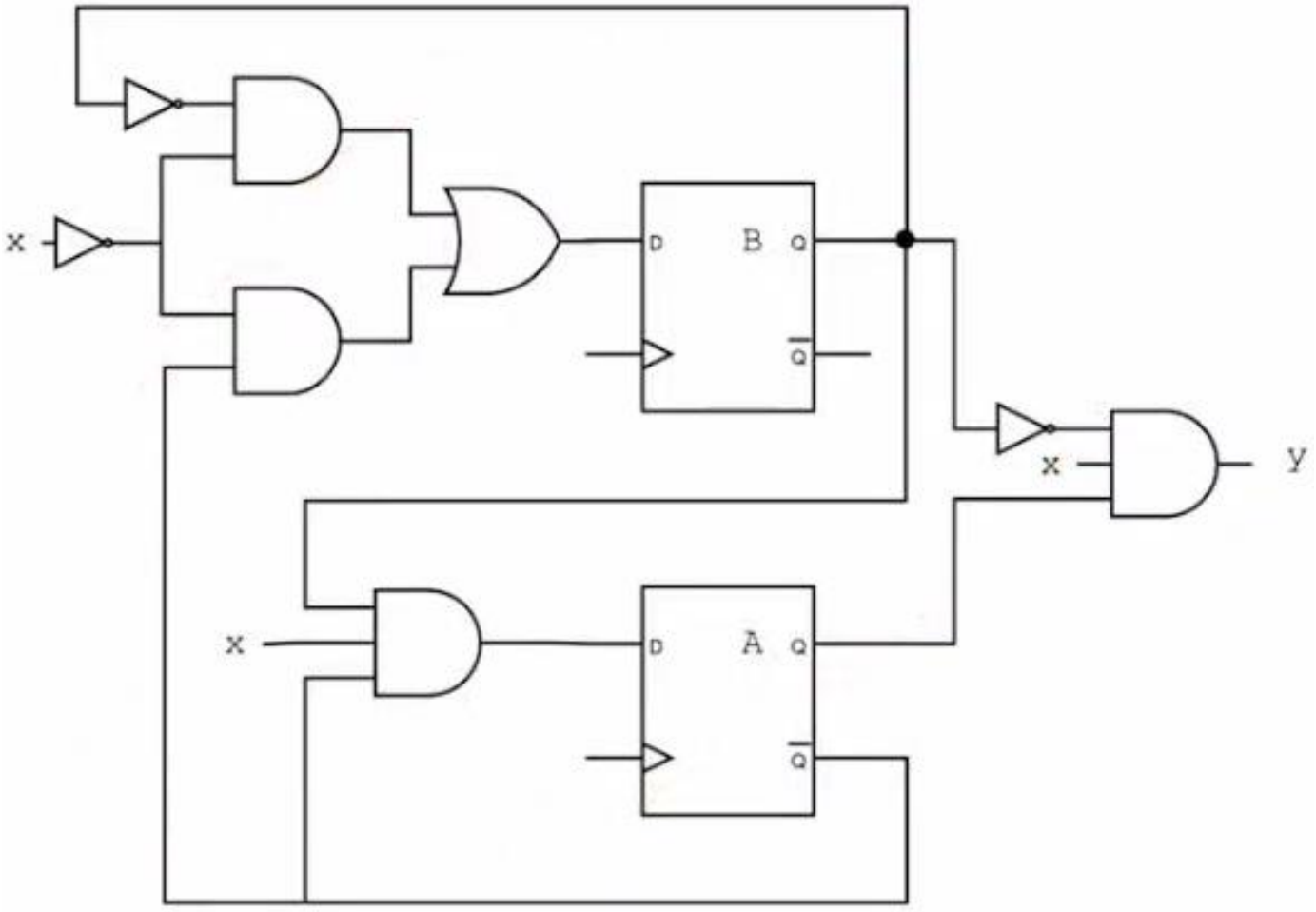
	Present State		Next State X = 0		Next State X = 1		Output
	A	B	A	B	A	B	Y
S_0	0	0	0	1	0	0	0
S_1	0	1	0	1	1	0	0
S_2	1	0	0	1	1	1	0
S_3	1	1	0	1	0	0	1

0 1 1 Sequence Detector





Example 2

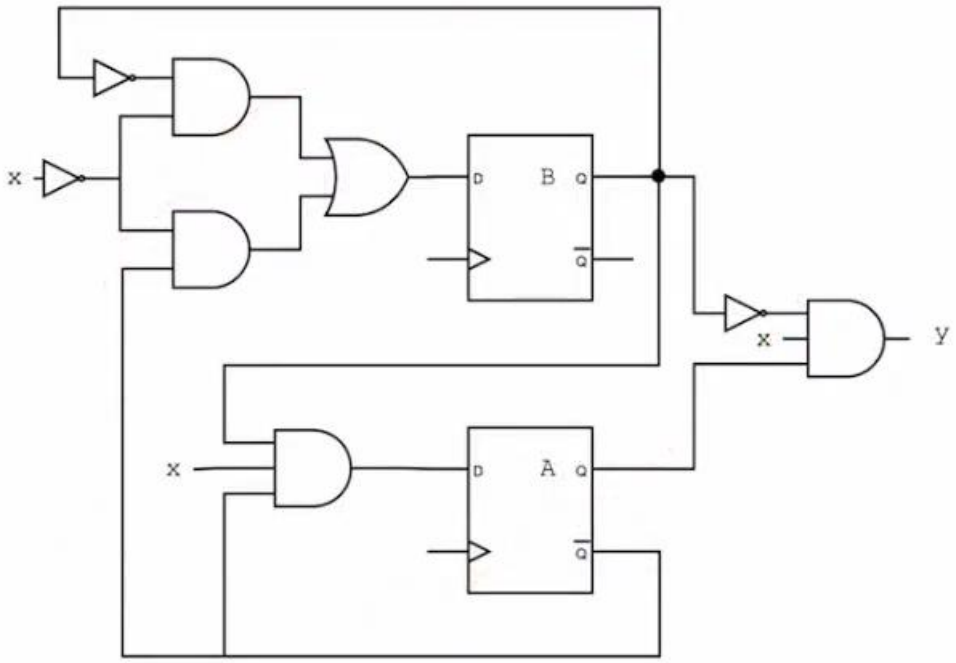


State Equation

$$A(t+1) = \bar{A}\,B\,X$$

$$B(t+1) = \bar{B}\,\bar{X} + \bar{A}\,\bar{X}$$

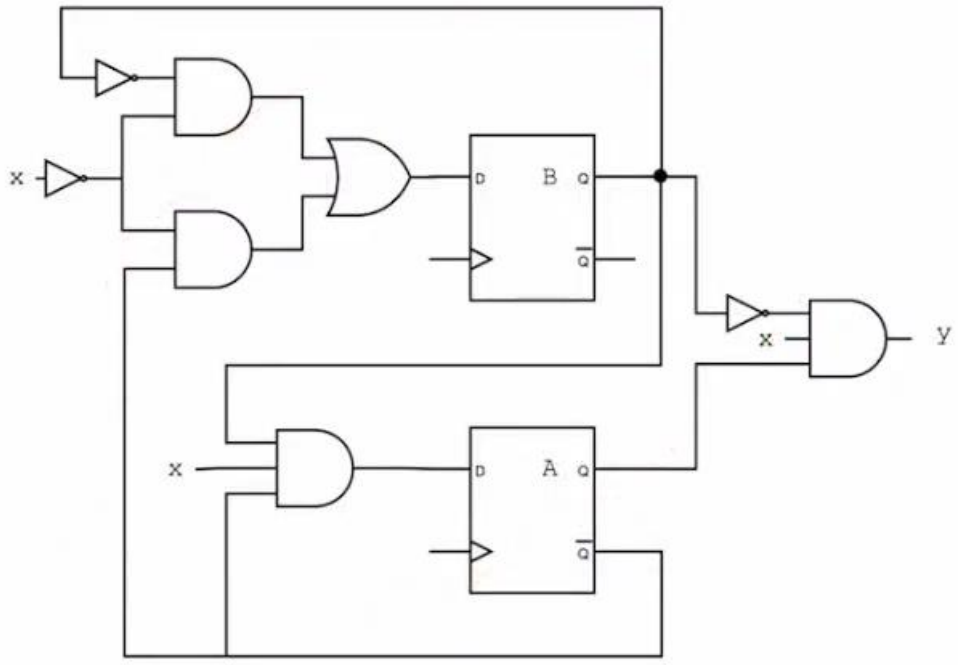
$$y = A\,\bar{B}\,X$$



State Table

$$A(t+1) = \bar{A}BX$$
$$B(t+1) = \bar{B}\bar{X} + \bar{A}\bar{X}$$
$$y = A\bar{B}X$$

Present State		Input	Next State		Output
A	B	X	A	B	Y



State Table

$$A(t+1) = \bar{A}BX$$
$$B(t+1) = \bar{B}\bar{X} + \bar{A}\bar{X}$$
$$y = A\bar{B}X$$

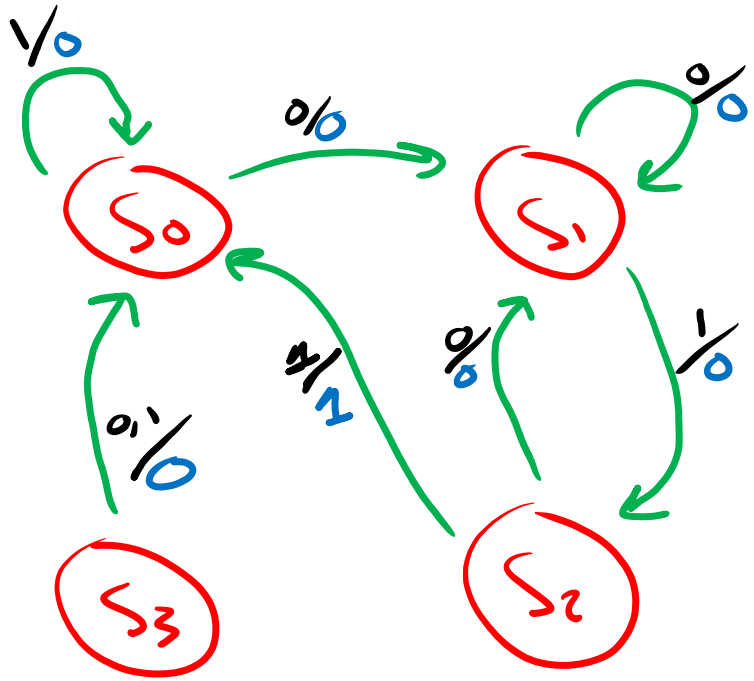
Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	1	0	0	1
1	1	0	0	0	0
1	1	1	0	0	0

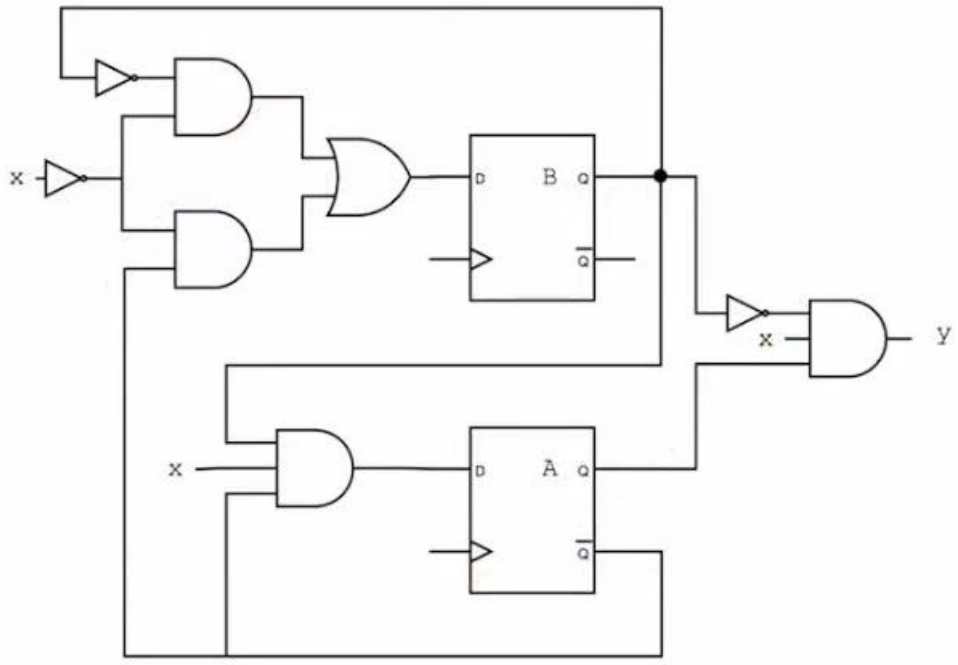
Present State		Next State X = 0		Next State X = 1		Output X = 0	Output X = 1
A	B	A	B	A	B	Y	Y

State Diagram

Present State		Next State X = 0		Next State X = 1		Output X = 0	Output X = 1
A	B	A	B	A	B	Y	Y
0	0	0	1	0	0	0	0
0	1	0	1	1	0	0	0
1	0	0	1	0	0	0	1
1	1	0	0	0	0	0	0

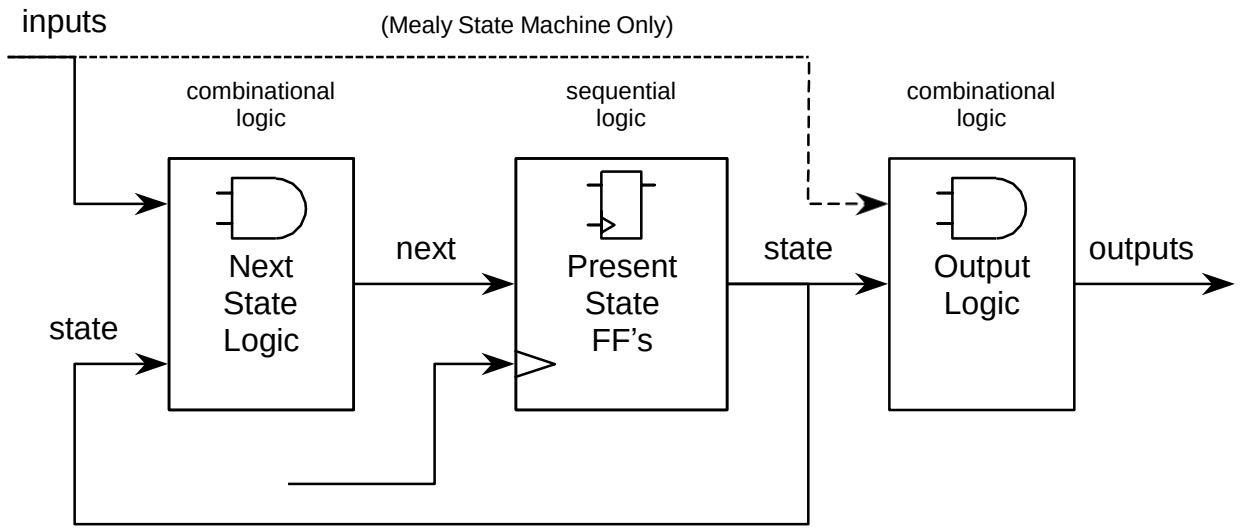
0 1 1 Sequence Detector





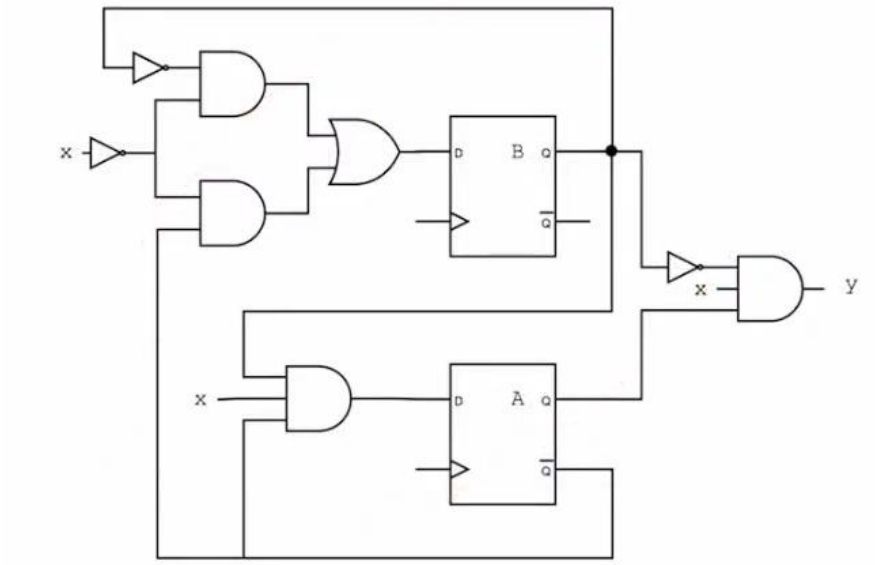
FSM Block Diagram

FSM Block Diagram

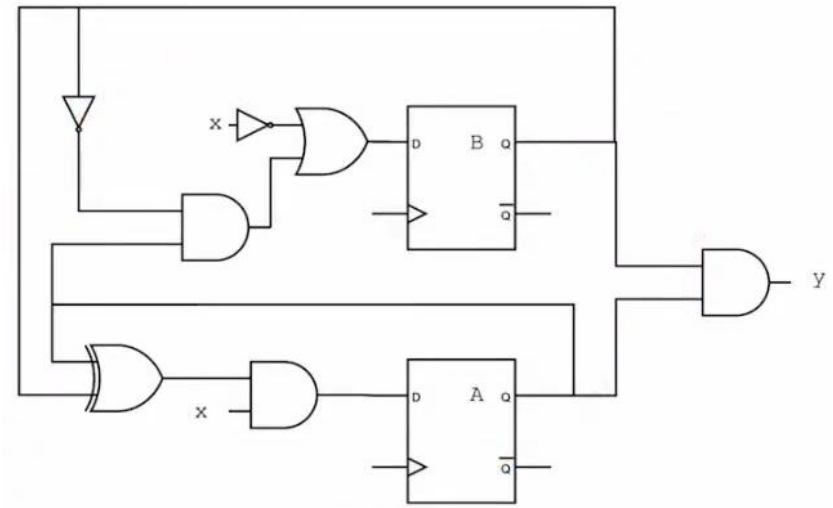


Mealy & Moore FSM

There are two types of state machines as classified by the types of outputs generated from each. The first is the Moore State Machine where the outputs are only a function of the present state, the second is the Mealy State Machine where one or more of the outputs are a function of the present state and one or more of the inputs.



Mealy FSM



Moore FSM

FSM

Moore machines are very useful because their output signals are synchronized with the clock. No matter when input signals reach the Moore Machine, its output signals will not change until the rising edge of the next clock cycle. This is very important to avoid setup timing violations. For example, if a Mealy machine's input signal(s) changes sometime in the middle of a clock cycle, one or more of its outputs and next state signals may change some time later. "Some time later" might come after the setup time threshold for the next rising edge. If this happens, the registers that will hold the FSMs next state may receive garbage, or just incorrect inputs. Obviously, this amounts to a bug(s) in your FSM. A very painful and difficult-to-find bug at that.

The tradeoff in using the Moore machine is that sometimes the Moore machine will require more states to specify its function than the Mealy machine. This is because in a Moore machine, output signals are only dependent on the current state. In a Mealy machine, outputs are dependent on both the current state and the inputs. The Mealy machine allows you to specify different output behavior for a single state

FSM Design Steps

1. Draw the state diagram based on specifications and desired behavior.
2. Reduce number of states if necessary. (This may make the combinational logic that drives the state registers more complex)
3. Assign binary values to states.(State Encoding)
4. Obtain the binary-coded state table.
5. Choose flip-flop type.
6. Get simplified flip-flop input equations and output equations.
7. Draw circuit diagram.

FSM Encoding Types

FSM Encoding Types

In as FSM design, each state is represented by a binary code, which are used to identify the state of the machine. These codes are the possible values of the state register. The process of assigning the binary codes to each state is known as state encoding.

The choice of encoding plays a key role in the FSM design. It influences the complexity, size, power consumption, speed of the design. If the encoding is such that the transitions of flip-flops (of state register) are minimized then the power will be saved. The timing of the machine are often affected by the choice of encoding.

The choice of encoding depends on the type of technology used like ASIC, FPGA, CPLD etc. and also the design specifications.

The following are the most common state encoding techniques used:

- **Binary encoding:** states are represented with binary encoded numbers: "000", "001", "010", "011", "100"...
- **One-hot encoding:** states are represented as bit patterns with exactly 1 '1': "000001", "000010", "000100", "001000", "010000"...
- **Gray coding:** the encoding of successive states only differ by one bit: "000", "001", "011", "010", "110"...

FSM Encoding Types

Binary Encoding: The code of a state is simply a binary number. The number of bits is equal to log2(N) rounded to next natural number.

- Suppose N = 6, then the number of bits are 3, and the state codes are:

  S0 - 000

  S1 - 001

  S2 - 010

  S3 - 011

  S4 - 100

  S5 – 101

- **Pros:**
  - Requires fewer flip-flops than other encodings (log2(N) for N states).
- **Cons:**
  - Many bits can switch at a time, which consumes more switching power.
  - Can lead to more complex next-state logic expressions.

FSM Encoding Types

One-hot Encoding: only one bit of the state vector is asserted for any given state. All other state bits are zero. Thus, if there are N states then N state flip-flops are required. As only one bit remains logic high and rest are logic low, it is called as One-hot encoding.

- If N = 5, then the number of bits (flip-flops) required are 5, and the state codes are:

  S0 - 00001

  S1 - 00010

  S2 - 00100

  S3 - 01000

  S4 – 10000

- **Pros:**
  - Simplifies next-state logic due to direct state representation.
  - No decoding logic needed for current state.
  - Faster state transitions due to parallel comparisons.
- **Cons:**
  - Requires more flip-flops (one per state).
  - Can increase logic resource usage in larger designs.
  - Susceptible to glitches on multiple flip-flops changing simultaneously.

FSM Encoding Types

Gray Encoding: uses the Gray codes, also known as reflected binary codes, to represent states, where two successive codes differ in only one digit. This helps is reducing the number of transition of the flip-flops outputs. The number of bits is equal to log2(N) rounded to next natural number.

- If N = 4, then 2 flip-flops are required, and the state codes are:

  S0 - 00

  S1 - 01

  S2 - 11

  S3 – 10

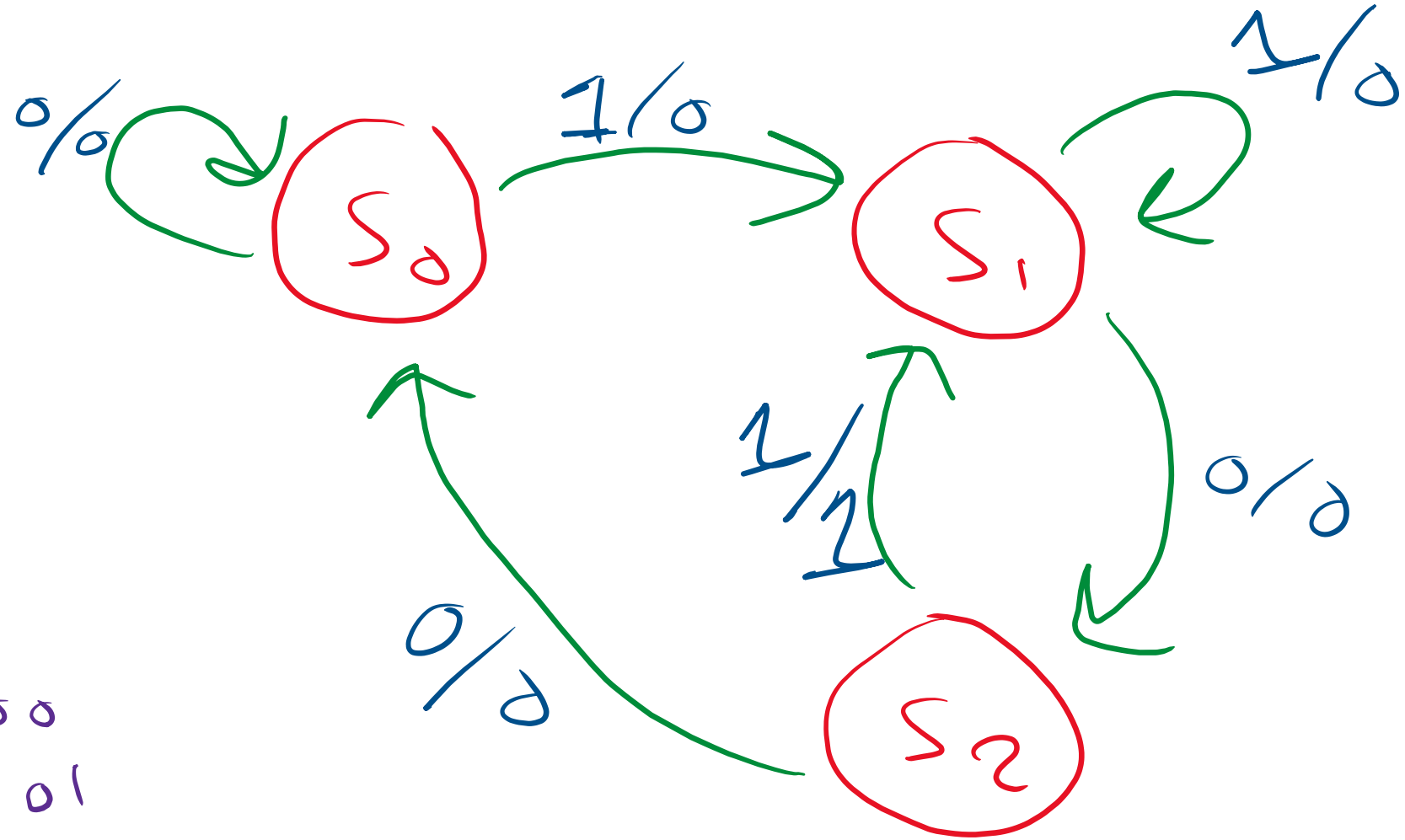
- **Pros:**
  - Reduces switching activity during state transitions (adjacent states differ by only one bit).
  - Improves reliability for asynchronous outputs, making them less sensitive to glitches.
- **Cons:**
  - Similar complexity to binary encoding for next-state logic.

Mealy 1 0 1 Sequence Detector Design

1 0 1 Sequence Detector Design

x	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0
y	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0
Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Mealy State Diagram

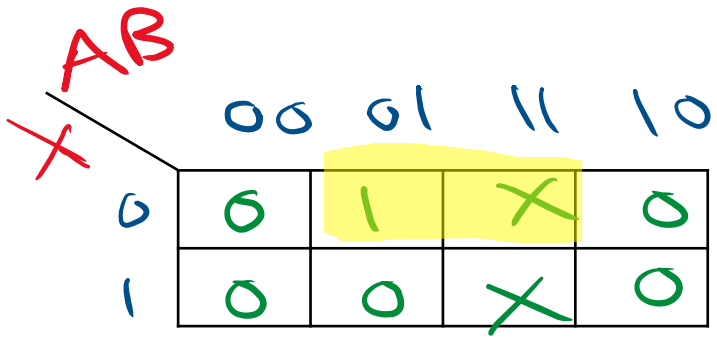


$S_0 \rightarrow 00$
$S_1 \rightarrow 01$
$S_2 \rightarrow 10$

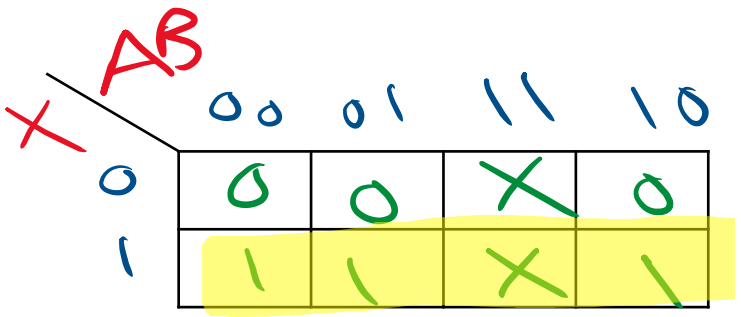
Mealy State Table

Present State		Next State X = 0		Next State X = 1		Output X = 0	Output X = 1
A	B	A	B	A	B	Y	Y
0	0	0	0	0	1	0	0
0	1	1	0	0	1	0	0
1	0	0	0	0	1	0	1
1	1	X	X	X	X	X	X

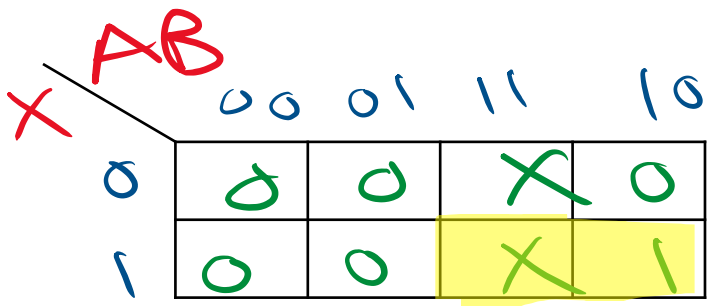
Mealy State Table



$$A(\tau+1) = B\bar{X}$$

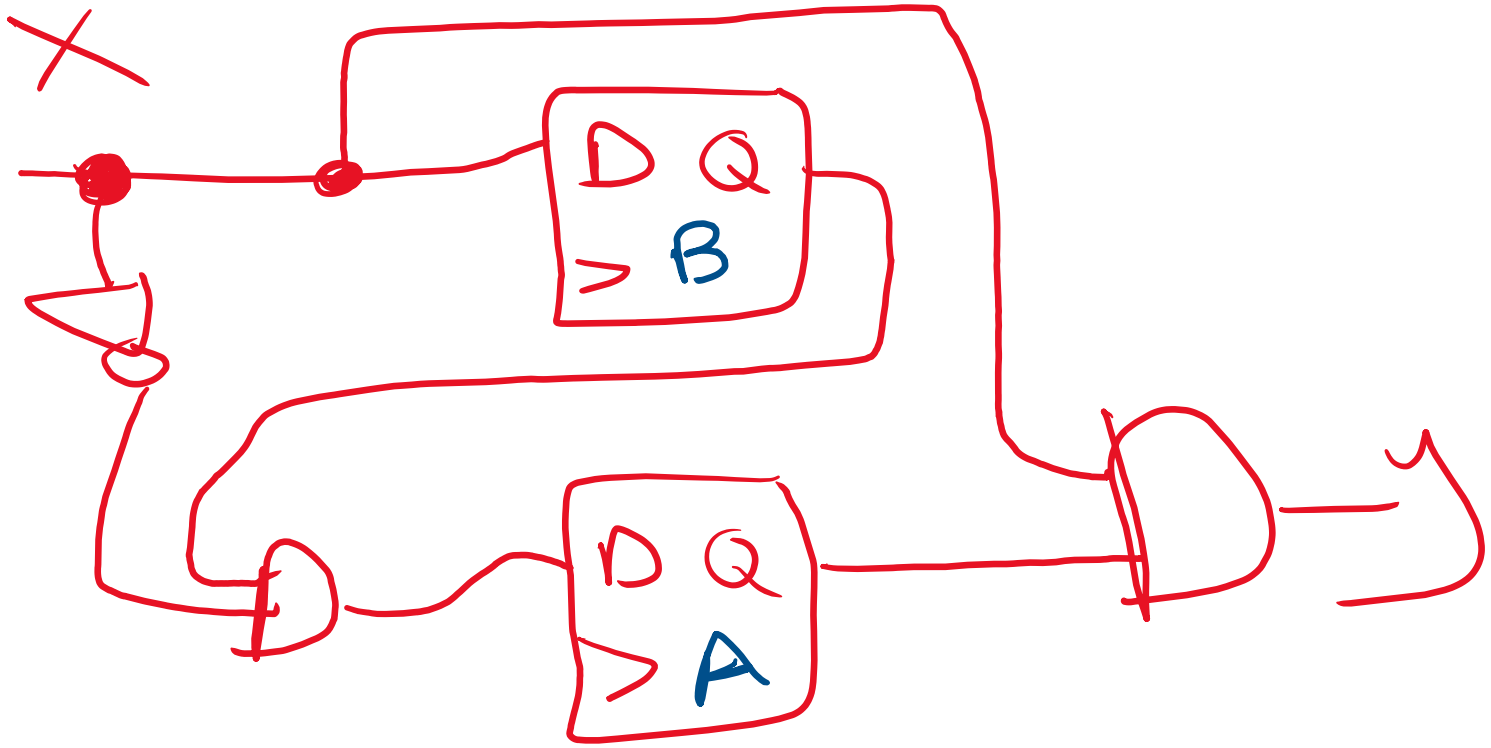


$$B(\tau+1) = X$$



$$y = AX$$

Mealy Hardware Circuit

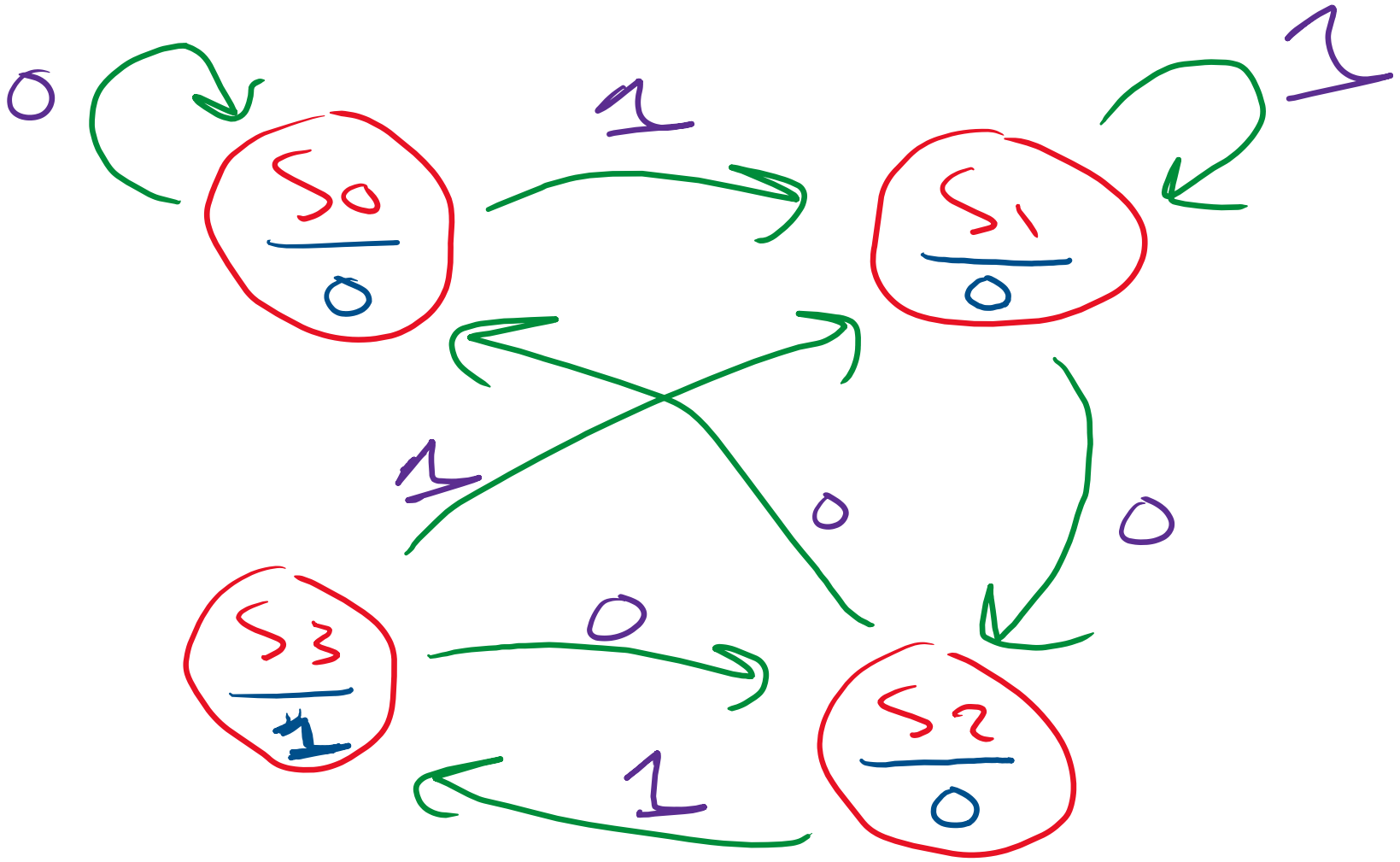


Moore 1 0 1 Sequence Detector Design

1 0 1 Sequence Detector Design

x	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0	
y		0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0
Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

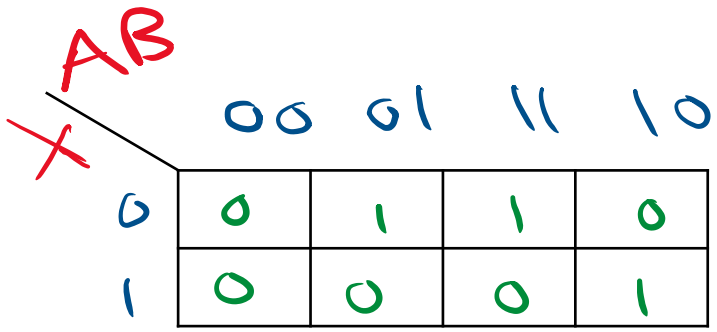
Moore State Diagram



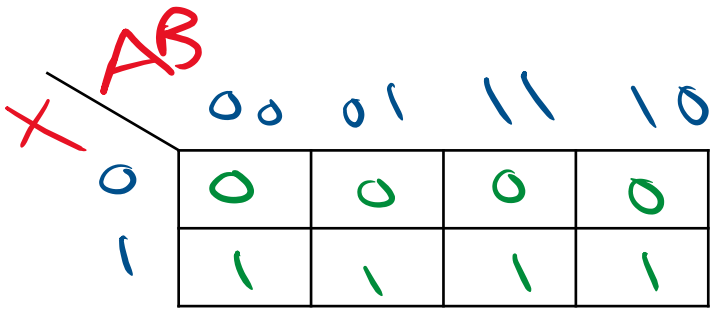
Moore State Table

Present State		Next State X = 0		Next State X = 1		Output
A	B	A	B	A	B	Y
0	0	0	0	0	1	0
0	1	1	0	0	1	0
1	0	0	0	1	1	0
1	1	1	0	0	1	1

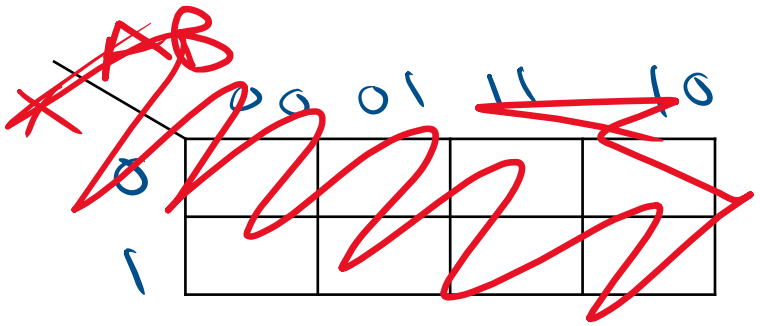
Mealy State Table



$$A(\tau+1) = B\bar{X} + A\bar{B}X$$

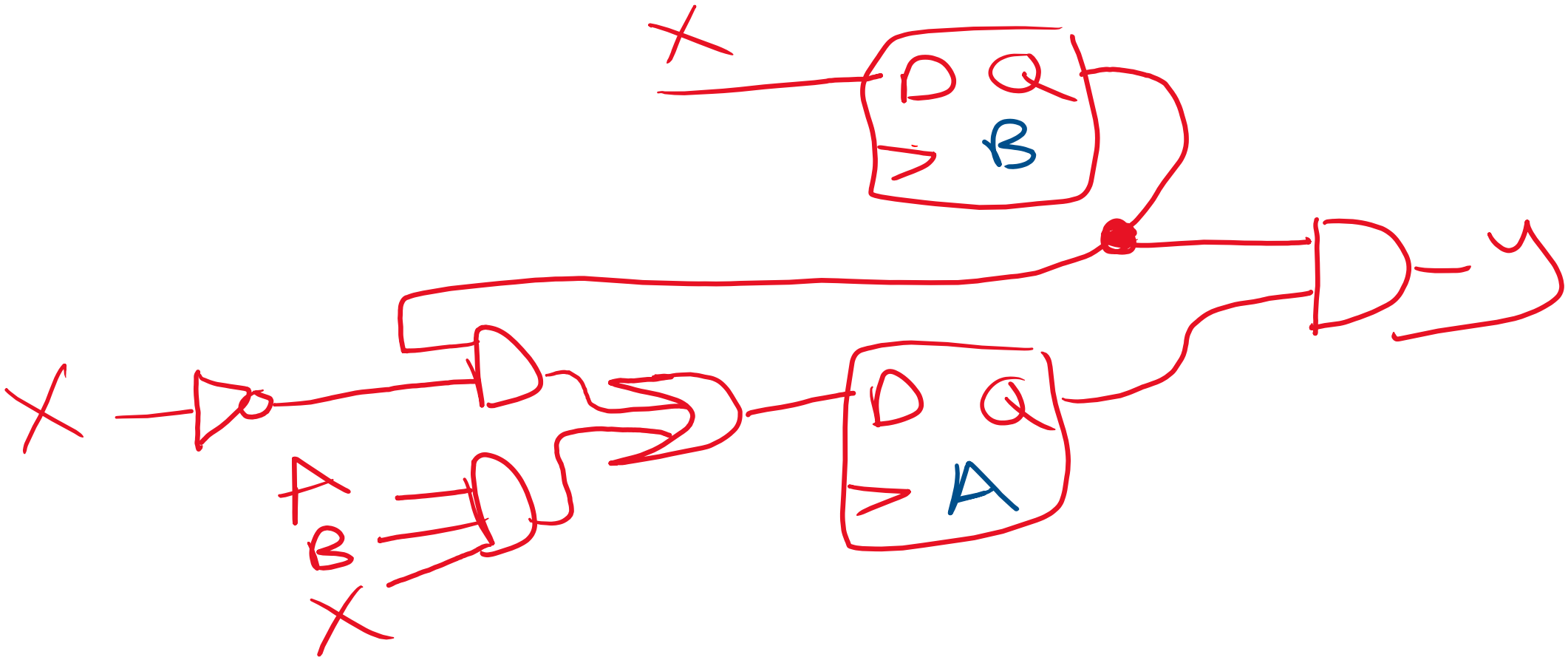


$$B(\tau+1) = X$$



$$Y = AB$$

Moore Hardware Circuit



Real Design Method

FSM Design Steps

1. Draw the state diagram based on specifications and desired behavior.
2. Reduce number of states if necessary. (This may make the combinational logic that drives the state registers more complex)
3. Assign binary values to states.
4. Obtain the binary-coded state table.
5. Choose flip-flop type.
6. Get simplified flip-flop input equations and output equations.
7. Draw circuit diagram.

Design Approaches

FSM with One always Block

- Combines state register and output logic in one block.

```
module fsm_one_block (input clk, rst_n, input in, output reg out);
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
  reg [1:0] current_state;
  //current state and output logic
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      current_state <= S0;
      out <= 0;
    end
else begin
    //current state
      case (current_state)
        S0: begin
          if (in)
            current_state <= S1;
        end
        S1: begin
          if (~in)
            current_state <= S2;
        end
        S2: current_state <= S0;
      endcase

    //output
      out <= (current_state == S2);
    end
  end
endmodule
```

FSM with Two always Blocks

- Separates state register and next state logic for clarity.
- Output logic generates in the current state always block in case of Moore FSM, and in the next state always block in case of Mealy FSM

```
module fsm_two_block (input clk, rst_n, input in,
output reg out);
   parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
   reg [1:0] current_state, next_state;

//current state and output logic
   always @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
         current_state <= S0;
         out <= 0;
      end else begin
         current_state <= next_state;
       out <= (current_state == S2);
      end
   end
```

```
//next state logic
   always @(*) begin
      next_state = S0;
      case (current_state)
         S0: begin
               if (in)
                  next_state = S1;
               end
         S1: begin
               if (~in)
                  next_state = S2;
               end
         S2: next_state = S0;
      endcase
end
endmodule
```

FSM with Two always Blocks

- Separates state register, next state logic and output logic for clarity.

```
module fsm_three_block (input clk, rst_n, input in, output reg out);
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
  reg [1:0] current_state, next_state;

//current state
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      current_state <= S0;
    end else begin
      current_state <= next_state;
    end
  end
```

```
//next state logic
  always @(*) begin
    next_state = S0;
    case (current_state)
      S0: begin
            if (in)
              next_state = S1;
          end
      S1: begin
            if (~in)
              next_state = S2;
          end
      S2: next_state = S0;
    endcase
end

//Output logic
  always @(*) begin
    out = (state == S2);
  end
endmodule
```

Thank You